

AIAC-ASSIGNMENT-9.5

NAME:E.ANJANI

BATCH:32

HNO:2303A52419

Problem 1: String Utilities Function Consider the following Python function:

```
def reverse_string(text):  
    return text[::-1]
```

Task:

1. Write documentation in:

- o (a) Docstring
- o (b) Inline comments
- o (c) Google-style documentation

```
Task1.py > ...  
1  def reverse_string(text: str) -> str:  
2      """  
3          This function takes a string as input and returns the reversed version of that string.  
4          Parameters:  
5          text (str): The string to be reversed.  
6          Returns:  
7          str: The reversed string.  
8          Raises:  
9          ValueError: If the input is not a string.  
10         """  
11         if not isinstance(text, str):  
12             raise ValueError("Input must be a string.")  
13         return text[::-1] # This slicing method reverses the string by starting from the end and moving to the beginning  
14  
15  
16 import Task1  
17 help(Task1)  
18
```

```
PS D:\courses\AIAC\Lab_ass_9.5> & c:\python314\python.exe d:/courses/AIAC/Lab_ass_9.5/Task1.py  
Help on module Task1:  
  
NAME  
    Task1  
  
FUNCTIONS  
    reverse_string(text: str) -> str  
        This function takes a string as input and returns the reversed version of that string.  
        Parameters:  
        text (str): The string to be reversed.  
-- More --
```

2. Compare the three documentation styles.

Style	Advantages
Docstring	Simple, readable, standard
Inline Comments	Easy to understand line-by-line
Google Style	Highly structured, professional

3. Recommend the most suitable style for a utility-based string library.

Google-style documentation is most suitable for a utility-based string library because it is structured, scalable, and widely used in professional projects.

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):  
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.

```
Task2.py > ...  
1  def check_strength(password: str) -> bool:  
2      """  
3          This function checks if the given password is strong enough based on its length.  
4          Parameters:  
5          password (str): The password to be checked.  
6          Returns:  
7          bool: True if the password is at least 8 characters long, otherwise False.  
8          Raises: ValueError: If the input is not a string.  
9      """  
10     if not isinstance(password, str):  
11         raise ValueError("Input must be a string.")  
12     return len(password) >= 8 # A password is considered strong if it has at least 8 characters.  
13  
14 import Task2  
15 help(Task2)  
16
```

2. Compare documentation styles for security-related code.

Inline comments → minimal explanation

Docstring → moderate clarity

Google style → clear, auditable, security-friendly

3. Recommend the most appropriate style.

Google-style documentation is most appropriate for security-related code because it clearly explains rules and behavior.

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:

o `square(n)`

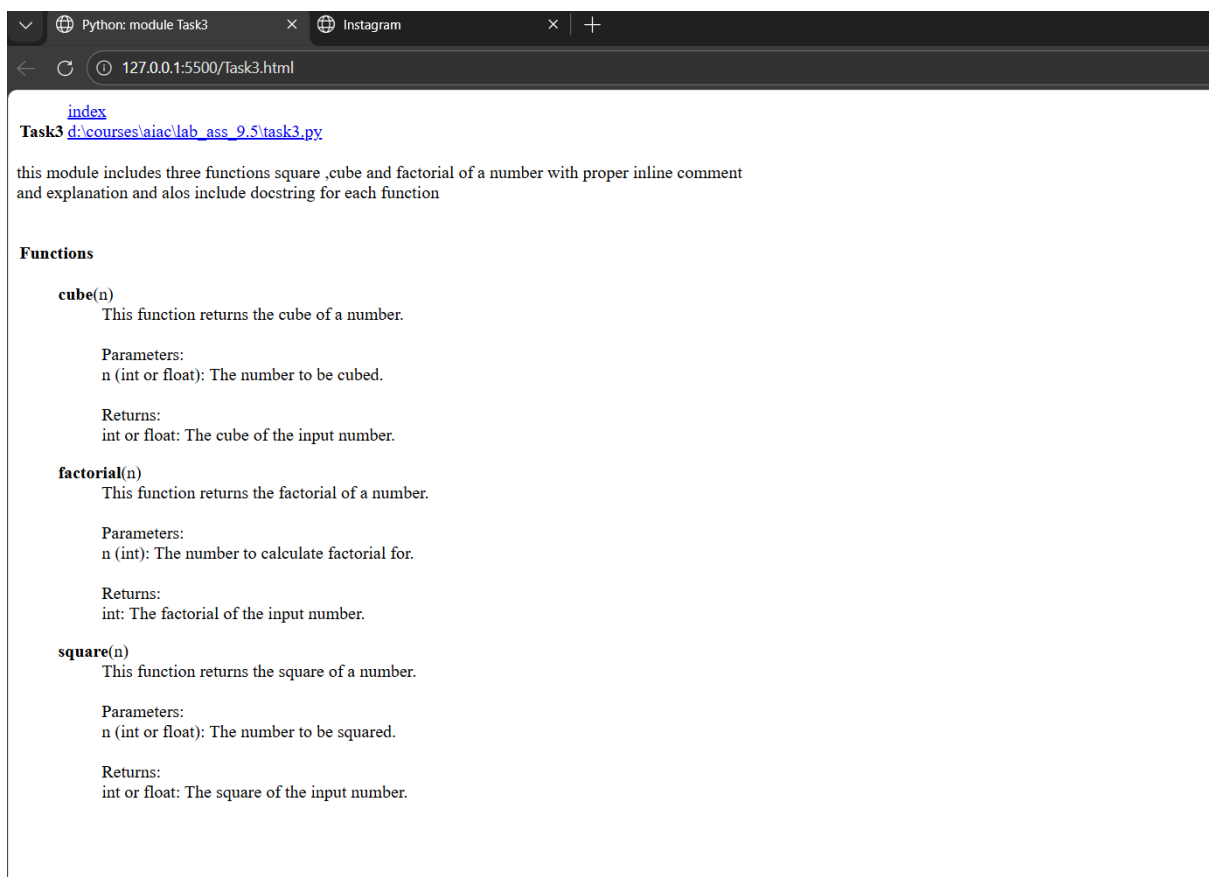
o `cube(n)`

o `factorial(n)`

2. Generate docstrings automatically using AI tools.

```
Task3.py > ...
1 """
2 this module includes three functions square ,cube and factorial of a number with proper inline comment
3 and explanation and alos include docstring for each function"""
4 def square(n):
5     """
6     This function returns the square of a number.
7
8     Parameters:
9     n (int or float): The number to be squared.
10
11     Returns:
12     int or float: The square of the input number.
13     """
14
15     return n * n # This line calculates the square of the input number by multiplying it by itself.
16 def cube(n):
17     """
18     This function returns the cube of a number.
19
20     Parameters:
21     n (int or float): The number to be cubed.
22
23     Returns:
24     int or float: The cube of the input number.
25     """
26
27     return n * n * n # This line calculates the cube of the input number by multiplying it by itself three times.
28 def factorial(n):
29     """
30     This function returns the factorial of a number.
31
32     Parameters:
33     n (int): The number to calculate factorial for.
34
35     Returns:
36     int: The factorial of the input number.
37     """
38
39     if n == 0:
40         return 1 # The factorial of 0 is defined to be 1, so we return 1 when n is 0.
41     else:
42         return n * factorial(n-1) # This line calculates the factorial of n by multiplying n with the factorial of (n-1) which is a recursive call to th
43
```

3. Export documentation as an HTML file.



Problem 4: Attendance Management Module

Task:

1. Create a module `attendance.py` with functions:

- o `mark_present(student)`
- o `mark_absent(student)`
- o `get_attendance(student)`

2. Add proper docstrings.

```
Task4.py > get_attendance
1 """
2 this module containf three functions mark_present(student),mark_absent(student),and get_attendance(student)
3 to manage student attendance with proper inline comment
4 and explanation and also include docstring for each function
5 """
6 attendance = {} # This dictionary will store the attendance status of each student.
7 def mark_present(student):
8     """
9     This function marks a student as present in the attendance dictionary.
10
11     Parameters:
12     student (str): The name of the student to be marked as present.
13
14     Returns:
15     None
16
17     """
18     attendance[student] = 'Present' # This line updates the attendance dictionary to mark the student as present.
19 def mark_absent(student):
20     """
21     This function marks a student as absent in the attendance dictionary.
22
23     Parameters:
24     student (str): The name of the student to be marked as absent.
25
26     Returns:
27     None
28
29     """
30     attendance[student] = 'Absent' # This line updates the attendance dictionary to mark the student as absent.
31 def get_attendance(student):
32     """
33     This function retrieves the attendance status of a student from the attendance dictionary.
34
35     Parameters:
36     student (str): The name of the student whose attendance status is to be retrieved.
37
38     Returns:
39     str: The attendance status of the student, which can be 'Present', 'Absent', or 'Not Marked' if the student is not in the dictionary.
40
41     """
42     return attendance.get(student, 'Not Marked') # This line retrieves the attendance status of the student from the dictionary, returning 'Not Marked'
```

3. Generate and view documentation in terminal and browse

```
PS D:\courses\AIAC\Lab_ass_9.5> python -m pydoc Task4
Help on module Task4:
```

NAME

Task4

DESCRIPTION

this module containf three functions mark_present(student),mark_absent(student),and get_attendance(student) to manage student attendance with proper inline comment and explanation and also include docstring for each function

FUNCTIONS

get_attendance(student)

This function retrieves the attendance status of a student from the attendance dictionary.

Parameters:

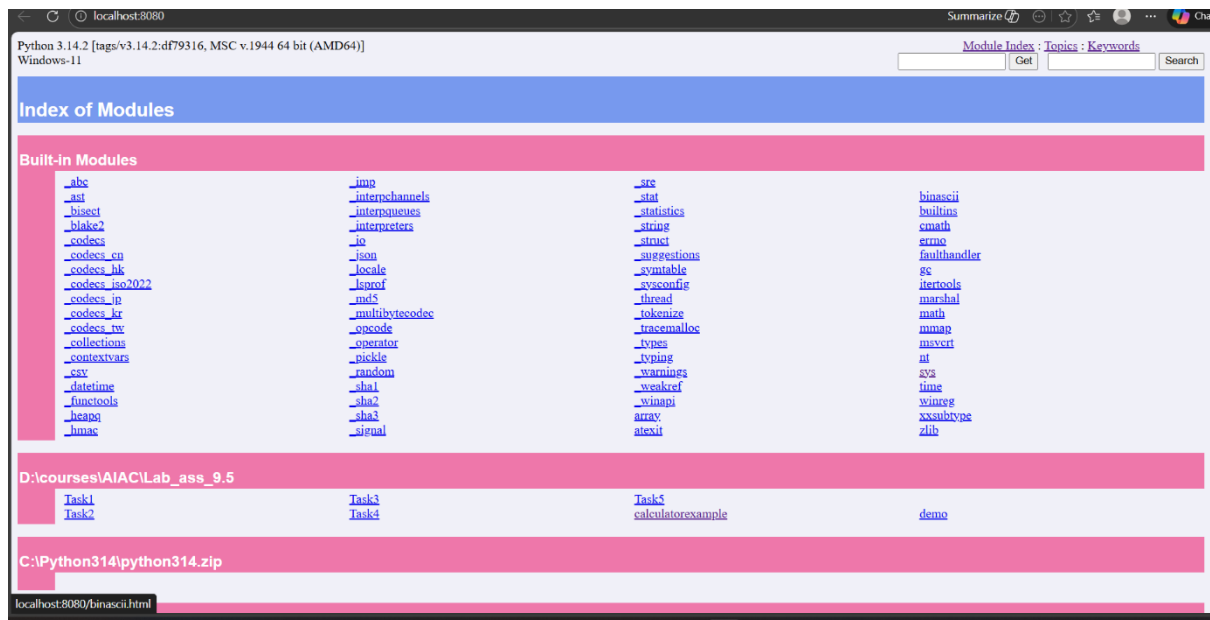
student (str): The name of the student whose attendance status is to be retrieved.

Returns:

str: The attendance status of the student, which can be 'Present', 'Absent', or 'Not Marked' if the student is not in the dict

ionary

```
PS D:\courses\AIAC\Lab_ass_9.5> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> |
```



Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

Task:

1. Write documentation using all three formats.

```

Task5.py > ...
1 def read_file(filename: str) -> str:
2     """
3     This function reads the contents of a file and returns it as a string.
4     Parameters:
5     filename (str): The name of the file to be read.
6     Returns:
7     str: The contents of the file as a string.
8     Raises:
9     ValueError: If the input is not a string.
10    FileNotFoundError: If the file does not exist.
11    """
12    if not isinstance(filename, str):
13        raise ValueError("Input must be a string.")
14    with open(filename, 'r') as f:
15        return f.read()# This line opens the specified file in read mode and returns its contents as a string.
16 import Task5
17 help(Task5)

```

2. Identify which style best explains exception handling.

- . Google-style documentation

3. Justify your recommendation.

- . Explicit Raises section
- . Clear explanation of errors
- . Best for debugging and maintenance